

Model Optimization

Making models smaller, faster, and cheaper to run — quantization, model distillation, pruning, small language models, inference and memory optimization, GPU vs. CPU inference, local LLMs, and on-device AI.

DIFFICULTY LEVEL

Advanced

ESTIMATED PRACTICE TIME

80-100 minutes

ESTIMATED READING TIME

50-65 minutes

PREREQUISITES

Modules 01-21, especially Module 21

LEARNING OUTCOMES

By the end of this module, you will be able to: explain quantization, distillation, and pruning as three distinct model-shrinking techniques; know when a small language model is the right choice; reason about inference and memory optimization tradeoffs; choose between GPU and CPU inference; and evaluate when local or on-device deployment makes sense.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Quantization
3.2	Model Distillation
3.3	Pruning
3.4	Small Language Models
3.5	Inference & Memory Optimization
3.6	GPU vs. CPU Inference
3.7	Local LLMs
3.8	On-Device AI
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

Module 21 covered specializing a model's behavior. This module covers making a model — specialized or not — smaller, faster, and cheaper to actually run, especially relevant for open-weight models (Module 11) you self-host.

Why is it important?

A model's raw capability is only half the story — how much memory, compute, and money it takes to actually run it in production determines whether a given deployment is even practical. Optimization techniques are what make running models on limited hardware, at the edge, or at massive scale economically viable.



Why should AI Engineers learn it?

Especially for teams working with open-weight models (Module 11), knowing how to shrink and speed up a model is what makes self-hosting economically and practically viable — connecting directly to Module 18's reliability/scalability and Module 09's cost optimization concerns.

Where is it used in the real world?

Mobile apps running AI features on-device, edge devices with limited compute, and any large-scale self-hosted deployment (Module 11, 3.4) trying to control infrastructure cost all depend on these optimization techniques.

Why is it relevant today?

As on-device and local AI (Sections 3.7-3.8) have grown from niche to mainstream — driven partly by privacy concerns and partly by cost — model optimization has become essential knowledge for a much wider range of AI engineering work than it was even recently.

Note

If you exclusively use closed, managed provider APIs (Module 12), most of this module's techniques (quantization, pruning) are handled entirely by the provider. This module is most directly actionable for teams working with open-weight models (Module 11, 3.4).

2. Before You Begin

RECAP

QLoRA (Module 21, 3.7) — already introduced quantization in a fine-tuning context; this module explores quantization as a standalone technique applicable beyond fine-tuning.

RECAP

Open-weight models (Module 11, 3.4) — most of this module's techniques apply specifically when you have access to a model's actual weights, not a closed API.



Tip

These techniques all involve tradeoffs (usually speed/size vs. quality). Always measure the quality impact on your actual task (Module 16) after applying any optimization — don't assume it's "free."

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 Quantization

DEFINITION

Quantization is reducing the numerical precision of a model's weights (e.g., from 16-bit to 4-bit numbers), shrinking its memory footprint and often speeding up inference, at some cost to precision.

SIMPLE EXPLANATION

A model's weights are just numbers; quantization stores those numbers with less precision (fewer bits each), making the whole model dramatically smaller in memory — often with surprisingly little quality loss, since LLMs tend to be fairly robust to reduced numerical precision.

 Like rounding prices to the nearest dollar instead of tracking exact cents — you lose a small amount of precision, but the overall information (roughly how much things cost) stays almost entirely intact, while the numbers become much simpler to work with.

EVERYDAY EXAMPLE

A model that requires 32GB of memory at full precision might run in under 8GB after 4-bit quantization — the difference between needing a specialized server and running on a high-end consumer GPU.

WHY IT MATTERS

Quantization is often the single highest-leverage technique for making a large open-weight model practical to run on limited hardware, directly connecting to Module 21's QLoRA and this module's local/on-device sections.

COMMON MISTAKES

Quantizing too aggressively for a task with tight quality requirements without testing the actual quality impact (Module 16) first.

BEST PRACTICES

Test quantized model quality against your actual evaluation set (Module 16) before deploying; start with moderate quantization levels and only go more aggressive if quality holds.

SUMMARY

Quantization shrinks a model's memory footprint by reducing numerical precision — often the highest-leverage single technique for practical self-hosted deployment.

3.2 Model Distillation

DEFINITION

Model distillation is training a smaller "student" model to mimic the outputs/behavior of a larger "teacher" model, transferring much of the larger model's capability into a more efficient package.

SIMPLE EXPLANATION

Rather than shrinking an existing model's own weights (quantization, pruning), distillation trains an entirely new, smaller model from scratch, using the larger model's outputs as training signal — similar in spirit to Module 21's SFT, but where the "labels" come from another AI model rather than humans.

 Like an apprentice learning by studying and imitating a master craftsman's finished work — the apprentice

(student model) develops real skill by learning from the master's (teacher model's) output, without needing the master's exact decades of full training.

EVERYDAY EXAMPLE

Training a small, fast model specifically to replicate a much larger model's classification decisions on a narrow task, achieving much of the larger model's accuracy at a fraction of its size and cost.

WHY IT MATTERS

Distillation can produce genuinely capable small models for narrow tasks, offering a different tradeoff than quantization (which shrinks an existing model rather than training a new one).

COMMON MISTAKES

Expecting a distilled model to match the teacher's full general capability across every possible task, rather than the specific narrow behavior it was actually distilled for.

BEST PRACTICES

Use distillation when you need a genuinely small, fast model for a specific, well-defined task, and have access to (or can generate) high-quality teacher-model outputs to train on.

SUMMARY

Distillation trains a new, smaller model to mimic a larger one's behavior — a different approach from shrinking an existing model directly, well-suited to narrow, well-defined tasks.

✂ 3.3 Pruning

DEFINITION

Pruning is removing parts of a model (individual weights or entire structural components) that contribute little to its output, reducing size and compute without a full retraining/distillation process.

SIMPLE EXPLANATION

Not every parameter in a large model contributes equally to its output — pruning identifies and removes the least important ones, shrinking the model directly, similar in spirit to quantization's goal (smaller, faster) but achieved by removal rather than reduced precision.

 *Like pruning a tree's dead or unproductive branches — the tree becomes more efficient and often healthier, having removed the parts contributing least to its overall growth, without needing to grow an entirely new tree.*

EVERYDAY EXAMPLE

A pruned version of a model that removes 30% of its least-impactful weights, achieving meaningfully faster inference with only a small, carefully-measured quality tradeoff.

WHY IT MATTERS

Pruning offers another lever (alongside quantization) for shrinking a model, and the two techniques can often be combined for compounding benefits.

COMMON MISTAKES

Pruning without careful evaluation of which components are actually safe to remove — over-aggressive pruning can cause disproportionate quality loss.

BEST PRACTICES

Prune incrementally with evaluation (Module 16) at each step; consider combining moderate pruning with quantization rather than pushing either technique to its extreme alone.

SUMMARY

Pruning removes low-impact parts of a model directly — a complementary technique to quantization, both aimed at a smaller, faster model.

🔗 3.4 Small Language Models

DEFINITION

Small Language Models (SLMs) are LLMs with meaningfully fewer parameters than today's largest flagship models, trading some general capability for dramatically lower cost, latency, and resource requirements.

SIMPLE EXPLANATION

This connects directly to Module 01, 3.12's reasoning-vs-chat framing and Module 09's cost optimization — for many tasks (simple classification, routing, straightforward extraction), a well-chosen small model performs adequately at a fraction of the cost of always defaulting to the largest available model.

💡 *Like hiring a focused specialist for a narrow, well-defined task instead of a highly-paid generalist expert — for the right task, the specialist does the job just as well at a fraction of the cost.*

EVERYDAY EXAMPLE

Using a small model to classify incoming support tickets by urgency (a narrow, well-defined task) while reserving a larger model for actually drafting nuanced responses — a form of Module 23's model routing.

WHY IT MATTERS

Small models are often the right choice for high-volume, narrow tasks — using a large flagship model everywhere is frequently unnecessary cost (Module 09, 3.4) for tasks that don't need that level of general capability.

COMMON MISTAKES

Assuming a small model is "worse" in every sense, rather than recognizing it's a deliberate tradeoff that's often the right one for narrow, well-defined tasks.

BEST PRACTICES

Test small models against your evaluation set (Module 16) for narrow, well-defined sub-tasks before assuming you need a larger model's full general capability.

SUMMARY

Small language models trade general capability for cost/speed — often the right choice for narrow, well-defined tasks rather than a strictly worse option.

🔗 3.5 Inference & Memory Optimization

DEFINITION

Inference optimization is speeding up the actual process of generating output from a model; memory optimization is reducing how much RAM/VRAM that process requires — often overlapping concerns, since memory bottlenecks frequently limit inference speed.

SIMPLE EXPLANATION

Beyond model-level techniques (quantization, pruning), inference optimization includes serving-level techniques — efficient batching of multiple requests, optimized attention computation, and careful memory management during generation — that speed up how a given model actually runs, without changing the model itself.

💡 *Like the difference between redesigning a car's engine (model-level: quantization, pruning) versus optimizing traffic flow and route planning for how that car actually gets used (serving-level: batching, memory management) — both make the overall trip faster, in different ways.*

EVERYDAY EXAMPLE

An inference server that efficiently batches multiple simultaneous user requests together, achieving much higher throughput than handling each request in complete isolation.

WHY IT MATTERS

Even a well-optimized (quantized, pruned) model can be served inefficiently — serving-level inference and memory optimization are a distinct, complementary layer of performance gains.

COMMON MISTAKES

Focusing exclusively on model-level optimization (quantization, pruning) while ignoring serving-level inefficiencies that may have equal or greater impact.

BEST PRACTICES

Consider both model-level (quantization, pruning) and serving-level (batching, memory management) optimization together for the full picture of inference performance.

SUMMARY

Inference and memory optimization address how efficiently a model is served, complementing model-level techniques like quantization and pruning.

3.6 GPU vs. CPU Inference

DEFINITION

This is the choice of hardware for running model inference — GPUs offer massive parallel compute well-suited to LLM workloads, while CPUs are more universally available but generally much slower for this specific task.

SIMPLE EXPLANATION

LLM inference involves enormous numbers of parallel matrix computations, which GPUs are specifically architected for — CPU inference is possible (especially for smaller, quantized models, Section 3.1) but generally significantly slower for anything beyond small models or low-throughput needs.

💡 *Like the difference between a factory with thousands of workers each doing one simple repetitive task in parallel (GPU) versus a handful of highly skilled generalists doing everything sequentially (CPU) — for the specific kind of repetitive, parallel work LLM inference requires, the factory model wins decisively.*

EVERYDAY EXAMPLE

Running a large, unquantized model on CPU might take many seconds per response, while the same model on a capable GPU responds in a fraction of a second — often an order-of-magnitude difference.

WHY IT MATTERS

Hardware choice is one of the most fundamental infrastructure decisions for self-hosted inference, directly affecting both cost and latency (Module 18).

COMMON MISTAKES

Assuming CPU inference is always impractical — for small, heavily quantized models with modest throughput needs, it can be a genuinely viable, much cheaper option.

BEST PRACTICES

Default to GPU inference for meaningful throughput/latency needs with larger models; consider CPU inference specifically for small, quantized models or genuinely low-volume use cases.

SUMMARY

GPUs generally dominate for LLM inference speed, but CPU inference remains viable for small, quantized models or low-throughput needs — a real infrastructure tradeoff to reason about deliberately.

🏠 3.7 Local LLMs

DEFINITION

Local LLMs are models run entirely on your own machine or infrastructure, rather than via a remote provider API (Module 12) — combining open-weight models (Module 11) with the optimization techniques in this module to make that practical.

SIMPLE EXPLANATION

Tools like Ollama and LM Studio package this entire workflow (downloading a quantized open-weight model, Section 3.1, and running it locally) into an accessible, simplified experience — no cloud API calls, no per-token cost, complete data privacy.

💡 *Like the difference between using a rental car service (cloud API) and owning your own car (local LLM) — no per-use fees and complete control, but you're responsible for maintenance and the upfront investment.*

EVERYDAY EXAMPLE

A developer running a quantized open-weight model via Ollama on their laptop for private, offline coding assistance, with zero API cost and complete data privacy.

WHY IT MATTERS

Local LLMs eliminate per-token cost and API dependency (Module 12) entirely, at the cost of hardware investment and generally lower raw capability than the largest closed-provider flagship models.

COMMON MISTAKES

Expecting a local, quantized, smaller model to match a large closed-provider flagship model's capability across every task.

BEST PRACTICES

Choose local LLMs deliberately for privacy-sensitive, offline, or cost-at-scale use cases; test capability against your actual task needs (Module 16) rather than assuming parity with cloud flagship models.

SUMMARY

Local LLMs trade some raw capability for privacy, offline access, and eliminated per-token cost — a deliberate, task-dependent choice rather than a strict downgrade.

🏠 3.8 On-Device AI

DEFINITION

On-device AI is running models directly on end-user hardware — phones, laptops, embedded devices — rather than on any server at all, local or remote.

SIMPLE EXPLANATION

This is local LLMs (Section 3.7) taken to its most constrained extreme — the hardware isn't a developer's powerful workstation but an end user's phone or embedded device, demanding the most aggressive optimization (heavy quantization, small models, Section 3.4) to fit within very tight resource constraints.

💡 *Like the difference between cooking in a full commercial kitchen (server-side/local LLM on a workstation) versus a camping stove (on-device) — the same basic goal, but working within dramatically tighter resource constraints demands a much more compact, efficient approach.*

EVERYDAY EXAMPLE

A phone's built-in AI features (text suggestions, on-device photo analysis) running entirely locally on the phone's own hardware, with no network call required — instant, private, and functional offline.

WHY IT MATTERS

On-device AI enables instant response times, complete offline functionality, and maximum privacy (data never leaves the device) — valuable properties no server-based approach, local or remote, can fully match.

COMMON MISTAKES

Attempting on-device deployment of a model without the aggressive optimization (heavy quantization, small model size) genuinely required for such constrained hardware.

BEST PRACTICES

Combine every technique in this module (quantization, pruning, small models) aggressively for genuine on-device deployment; test real-world performance on actual target devices, not just development hardware.

SUMMARY

On-device AI runs models directly on end-user hardware, demanding the most aggressive optimization of any deployment target — but offering instant response, offline function, and maximum privacy in return.

4. Visual Learning

🔧 Optimization Techniques by Approach

Model Optimization

— Shrink the Existing Model

Quantization, Pruning

— Train a New, Smaller Model

Distillation, Small Language Models

— Optimize How It's Served

Inference/Memory Optimization, GPU/CPU choice

📊 Deployment Spectrum

Cloud API
(Module 12)

→

Local LLM
(own workstation)

→

On-Device AI
(end-user hardware)

Least constrained

→

Most constrained

Least optimization needed

→

Most optimization needed

📊 Technique Comparison

Technique	What It Does	Typical Tradeoff
Quantization	Reduces numerical precision	Small quality loss, big size reduction
Pruning	Removes low-impact weights	Requires careful evaluation
Distillation	Trains a new smaller model	Narrower capability than teacher
Small language models	Fewer parameters from the start	Less general capability, much cheaper

5. Real World Applications

Product	How these fundamentals show up
Mobile apps with on-device AI features	Heavily quantized small models running directly on the phone.
Privacy-focused developer tools	Local LLMs via Ollama/LM Studio for offline, private coding assistance.
High-volume classification pipelines	Small language models handle narrow tasks at a fraction of flagship-model cost.
Self-hosted enterprise AI	Quantized open-weight models on GPU infrastructure, balancing cost and capability.

6. Practical Examples

Beginner: Choosing an optimization technique

For each: (1) you need a model on a memory-constrained laptop, (2) you need a narrow classifier at massive scale, (3) you need the fastest possible large-model inference — which technique(s) from Section 3 fit best?

Beginner: Local vs. cloud tradeoffs

List 3 scenarios favoring a local LLM over a cloud API, and 3 favoring the reverse.

Intermediate: Estimating quantization impact

Research the approximate memory footprint of a specific open-weight model at full precision vs. 4-bit quantization, and calculate the reduction.

Advanced: On-device deployment plan

Sketch a deployment plan for a hypothetical on-device AI feature, specifying which optimization techniques you'd combine and why.

7. Code Examples

Python — Conceptual: loading a quantized model (Hugging Face pattern)

```
# pip install transformers bitsandbytes accelerate
# Conceptual pattern — actual setup varies by model/hardware
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

# Configure 4-bit quantization (Section 3.1)
quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
)

model_name = "some-open-weight-model" # e.g. a Llama or Mistral variant
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quant_config,
    device_map="auto", # automatically uses GPU if available (Section 3.6)
)

# Same interface as an unquantized model — just far less memory required
inputs = tokenizer("Explain quantization in one sentence.", return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(outputs[0]))
```

Shell — Running a local LLM via Ollama

```
# Ollama packages model download, quantization, and local serving together
# Install Ollama, then:

ollama pull llama3 # downloads a pre-quantized model (Section 3.1, 3.7)
ollama run llama3 "Explain the water cycle in two sentences."

# Ollama also exposes a local API compatible with common client libraries,
# so your application code can call it much like a cloud provider (Module 12)
curl http://localhost:11434/api/generate -d '{
  "model": "llama3",
  "prompt": "Explain the water cycle in two sentences."
}'
```



Tip

Tools like Ollama and LM Studio deliberately hide most of this module's complexity (quantization format, GGUF conversion, hardware detection) behind a simple interface — a great starting point before diving into manual quantization configuration.

8. Common Mistakes

- **Quantizing too aggressively** without testing quality impact on your actual task.
- **Expecting a distilled model to match its teacher's full general capability** across every task, not just the narrow one it was distilled for.
- **Over-pruning without careful, incremental evaluation** at each step.
- **Assuming small models are strictly worse**, rather than a deliberate, often-correct tradeoff for narrow tasks.
- **Focusing only on model-level optimization** while ignoring serving-level inefficiencies.
- **Assuming CPU inference is always impractical**, missing genuinely viable small-model/low-throughput cases.
- **Attempting on-device deployment without sufficiently aggressive optimization** for the target hardware's real constraints.

9. Best Practices

- **Test optimized model quality against your real evaluation set** (Module 16) after any optimization technique.
- **Combine complementary techniques** (quantization + pruning, or quantization + a small model) rather than pushing one to its extreme alone.
- **Match hardware to actual throughput needs** — GPU for meaningful scale, CPU viable for small/low-volume cases.
- **Choose local/on-device deployment deliberately** for privacy, offline, or cost-at-scale needs, not as a default.
- **Test real-world performance on actual target hardware**, especially for on-device deployment.

10. Hands-on Activities

Easy 5 exercises

1. Explain, in your own words, the difference between quantization and pruning.
2. List 3 scenarios where a small language model is a good fit, and 2 where it isn't.
3. Sketch the optimization techniques tree from Section 4 from memory.
4. Explain why GPUs generally outperform CPUs for LLM inference.
5. List 3 benefits of on-device AI that no server-based approach can fully match.

Intermediate 3 exercises

1. Install Ollama (or a similar tool) and run a local quantized model, comparing its response quality to a cloud API for a simple task.
2. Research 2 real open-weight models and compare their available quantization options and resulting memory footprints.
3. Design a small-language-model routing strategy (connecting to Module 23) for a hypothetical multi-task application.

Challenge 2 exercises

1. Build the quantized model loading example from Section 7 (if you have suitable hardware) and measure the memory footprint difference vs. full precision.
2. Design a complete on-device AI deployment plan for a hypothetical mobile feature, specifying model size targets, quantization level, and expected performance on representative hardware.

11. Mini Project

Goal

Set up and evaluate a local LLM, comparing its practical performance and quality against a cloud API for a real task from earlier in this course.

Requirements

- Ollama or LM Studio installed locally
- An Anthropic API key (for comparison)
- A task/evaluation set from Module 16 or an earlier module

Step-by-Step Instructions

1. Install Ollama (or LM Studio) and download a quantized open-weight model.
2. Run your evaluation set (Module 16) against the local model, recording accuracy/quality scores.
3. Run the same evaluation set against a cloud API model for comparison.
4. Measure and compare: response latency, quality score, and (conceptually) cost (local = hardware cost, cloud = per-token cost).
5. Write a short recommendation: for this specific task, which deployment makes more sense, and why?

Expected Output

```
Task: [your evaluation task]

Local (quantized, via Ollama):
  Quality score: 3.6/5   Avg latency: 2.1s   Cost: hardware only

Cloud API:
  Quality score: 4.4/5   Avg latency: 1.3s   Cost: $0.002/request

Recommendation: For this task's quality bar, cloud API is worth
the per-token cost. Local model would fit better for a higher-
volume, lower-stakes task where the local model's score is sufficient.
```

Possible Improvements

- Test multiple quantization levels of the same local model to find the best quality/size tradeoff for your task.
- Try a distilled or purpose-built small model specifically for your narrow task and compare.
- Estimate real hardware cost (electricity, equipment) for the local option to make the cost comparison concrete.

12. Review Questions

Multiple Choice (10)

1. Quantization primarily works by:

- A) Removing weights entirely B) Reducing the numerical precision of a model's weights C) Training a new model D) Adding more parameters

2. Model distillation trains:

- A) The exact same model again B) A smaller "student" model to mimic a larger "teacher" model C) Nothing new D) Only the teacher model further

3. Pruning works by:

- A) Adding new parameters B) Removing low-impact weights or components C) Increasing precision D) Training from scratch

4. Small language models are best suited for:

- A) Every possible task equally B) Narrow, well-defined tasks where full general capability isn't needed C) Nothing useful D) Only image generation

5. Inference and memory optimization primarily address:

- A) The model's own weights B) How efficiently a model is served, at the infrastructure level C) Training data quality D) Prompt wording

6. GPUs generally outperform CPUs for LLM inference because:

- A) They're always cheaper B) They offer massive parallel compute well-suited to matrix operations C) CPUs can't run any AI workload D) GPUs use less power

7. Local LLMs are best described as:

- A) Always identical in capability to cloud flagship models B) Models run on your own machine/infrastructure instead of a remote API C) A type of cloud-only service D) Unrelated to open-weight models

8. On-device AI runs models:

- A) On a remote data center only B) Directly on end-user hardware like phones C) Nowhere, it's theoretical D) Only on desktop workstations

9. On-device AI generally requires:

- A) No optimization at all B) The most aggressive optimization of any deployment target C) Less optimization than cloud APIs D) Only pruning, nothing else

10. Tools like Ollama and LM Studio primarily simplify:

- A) Model training from scratch B) Downloading and running quantized models locally C) Cloud API billing D) Prompt engineering

Identification (5)

1. ____ reduces a model's numerical precision to shrink its memory footprint.

2. ____ trains a smaller model to mimic a larger model's behavior.

3. ____ removes low-impact weights or components from a model.

4. ____ are models run entirely on your own machine instead of a remote provider API.

5. ____ runs models directly on end-user hardware like phones or embedded devices.

True or False (5)

1. Quantization always causes severe, unacceptable quality loss.

2. A distilled model generally matches its teacher's narrow target behavior, not necessarily its full general capability.

3. CPU inference is never a practical option for any LLM workload.

4. Local LLMs eliminate per-token API cost but require hardware investment.

5. On-device AI generally requires more aggressive optimization than server-based deployment.

Situational (5)

1. You need to run a large open-weight model on a single consumer GPU with limited VRAM. What technique would you apply first?
2. You need a fast, narrow classifier at very high volume and want to minimize cost. What model category fits best?
3. You're building a privacy-sensitive app that must work fully offline. What deployment approach fits?
4. Your quantized model's quality dropped noticeably on your evaluation set. What should you do?
5. You want the absolute fastest possible inference for a large model at meaningful scale. What hardware choice fits best?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Quantization, 2-Model distillation, 3-Pruning, 4-Local LLMs, 5-On-device AI

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Quantization (e.g., 4-bit) to shrink the model's memory footprint.
2. A small language model, possibly via distillation for the specific narrow task.
3. Local LLM or on-device AI, depending on the target hardware.
4. Reduce quantization aggressiveness, or combine with less aggressive pruning; re-evaluate on your golden dataset.
5. GPU inference.

13. Cheat Sheet

Shrink Existing

Quantization — less precision.

Pruning — remove low-impact weights.

→ Train New/Smaller

Distillation — student mimics teacher.

SLMs — fewer parameters from the start.

Hardware

GPU for scale/speed. CPU viable for small, quantized, low-volume.

Deployment

Cloud → Local → On-Device: increasing constraint, increasing optimization need.

14. Key Takeaways

- Quantization and pruning shrink an existing model directly; distillation and small language models start smaller or train new, more efficient ones.
- Small language models are a deliberate, often-correct tradeoff for narrow, well-defined, high-volume tasks — not strictly inferior.
- Inference and memory optimization address serving-level efficiency, complementing model-level techniques.
- GPUs generally dominate LLM inference speed; CPU inference remains viable for small, quantized models or low-throughput needs.
- Local LLMs eliminate per-token cost and enable offline/private use, at the cost of hardware investment and generally lower raw capability.
- On-device AI demands the most aggressive optimization of any deployment target, in exchange for instant response, offline function, and maximum privacy.
- Always test optimized model quality against your real evaluation set — these techniques all involve real tradeoffs, never a free lunch.

15. Additional Resources

Tools

- Ollama and LM Studio — simplified local LLM deployment
- Hugging Face Transformers + bitsandbytes — quantization tooling
- llama.cpp and GGUF format — efficient local/CPU inference (also covered in Module 29)

Related Modules

- Module 21 — Model Fine-Tuning (QLoRA's use of quantization)
- Module 29 — Local & Open-Source AI (deeper dive into local tooling)
- Module 23 — Model Routing & Selection (choosing between small and large models per task)

Communities & Further Learning

- Open-weight model communities (Hugging Face, r/LocalLLaMA-style forums) for hands-on optimization guides and benchmarks.